LAB REPORT: CPU SCHEDULING ALGORITHMS

Course: Operating Systems

Student Name: Aryan Nautiyal **Roll Number:** 251210015

1. Aim

To implement and simulate CPU scheduling algorithms(FCFS, SJF, SRTF) using process arrival time and burst time, and to calculate and compare various performance metrics(CT,TAT,WT,RT,Average WT,Average TAT, Average RT).

2. Theory

Key formulas used in evaluation include:

$$\text{Turnaround Time (TAT)} = \text{Completion Time (CT)} - \text{Arrival Time (AT)}$$

$$\text{Waiting Time (WT)} = \text{Turnaround Time (TAT)} - \text{Burst Time (BT)}$$

$$\text{Response Time (RT)} = \text{First Execution Start Time} - \text{Arrival Time (AT)}$$

- **FCFS:** Processes are executed strictly in order of their arrival. Simple to implement, but suffers from the Convoy Effect where short processes wait behind long ones.
- **Non-Preemptive SJF:** Selects the process with the smallest initial burst time among all arrived processes. Once started, a process runs to completion.
- **SRTF (Preemptive SJF):** The process with the minimum remaining CPU burst time is selected for execution. A newly arrived process with a shorter remaining burst time will preempt the currently executing process.

3. Source Code

```
#include <iostream>
#include <vector>
#include <queue>
#include <string>
#include <iomanip>

enum class Policy { FCFS, SJF, SRTF };

struct Process {
    int id, at, bt, remaining_bt;
    int ct = 0, tat = 0, wt = 0, rt = -1;
    Process(int id, int at, int bt) : id(id), at(at), bt(bt), remaining_bt(bt) {}
};

class Scheduler {
private:
    std::string name;
    Policy policy;
    std::vector<Process> procs;

public:
    Scheduler(std::string name, Policy p, std::vector<Process> workload)
        : name(name), policy(p), procs(workload) {}

    void run() {
        auto comp = [this](const Process* a, const Process* b) {
            if (policy == Policy::SJF) return a->bt != b->bt ? a->bt > b->bt : a->at > b->at;
            if (policy == Policy::SRTF) return a->remaining_bt != b->remaining_bt ? a-
>remaining_bt > b->remaining_bt : a->at > b->at;
            return a->at > b->at; // FCFS
        };

        std::priority_queue<Process*, std::vector<Process*>, decltype(comp)> ready(comp);
        Process* running = nullptr;
```

```
        int tick = 0, completed = 0, n = procs.size();

        while (completed < n) {
            for (auto &p : procs) if (p.at == tick) ready.push(&p);

            if (policy == Policy::SRTF && running) { ready.push(running); running =
nullptr; }
            if (!running && !ready.empty()) { running = ready.top(); ready.pop(); }

            if (running) {
                if (running->rt == -1) running->rt = tick - running->at;
                if (--running->remaining_bt == 0) {
                    running->ct = tick + 1;
                    running->tat = running->ct - running->at;
                    running->wt = running->tat - running->bt;
                    completed++;
                    running = nullptr;
                }
            }
            tick++;
        }
    }

    void printReport() const {
        double sum_tat = 0, sum_wt = 0, sum_rt = 0;
        std::cout << name << "\n";
        std::cout << std::left
                  << std::setw(6) << "PID"
                  << std::setw(6) << "AT"
                  << std::setw(6) << "BT"
                  << std::setw(6) << "CT"
                  << std::setw(6) << "TAT"
                  << std::setw(6) << "WT"
                  << std::setw(6) << "RT" << "\n";

        for (const auto &p : procs) {
            std::cout << std::left
                      << std::setw(6) << ("P" + std::to_string(p.id))
                      << std::setw(6) << p.at
                      << std::setw(6) << p.bt
                      << std::setw(6) << p.ct
                      << std::setw(6) << p.tat
                      << std::setw(6) << p.wt
                      << std::setw(6) << p.rt << "\n";
            sum_tat += p.tat; sum_wt += p.wt; sum_rt += p.rt;
        }

        std::cout << "Avg TAT: " << std::fixed << std::setprecision(2) << (sum_tat /
procs.size())
                  << " | Avg WT: " << (sum_wt / procs.size())
                  << " | Avg RT: " << (sum_rt / procs.size()) << "\n\n";
    }
};

int main() {
    std::vector<Process> workload = {{1, 0, 8}, {2, 1, 4}, {3, 2, 9}, {4, 3, 5}};

    Scheduler fcfs("First Come First Serve (FCFS)", Policy::FCFS, workload);
    Scheduler sjf("Shortest Job First (SJF)", Policy::SJF, workload);
    Scheduler srtf("Shortest Remaining Time First (SRTF)", Policy::SRTF, workload);

    for (auto* s : {&fcfs, &sjf, &srtf}) {
```

```
        s->run();
        s->printReport();
    }
    return 0;
}
```

4. Terminal Output

```
$ g++ scheduler.cpp -o scheduler && ./scheduler

First Come First Serve (FCFS)
PID   AT    BT    CT    TAT   WT    RT
P1    0     8     8     8     0     0
P2    1     4     12    11    7     7
P3    2     9     21    19    10    10
P4    3     5     26    23    18    18
Avg TAT: 15.25 | Avg WT: 8.75 | Avg RT: 8.75

Shortest Job First (SJF)
PID   AT    BT    CT    TAT   WT    RT
P1    0     8     8     8     0     0
P2    1     4     12    11    7     7
P3    2     9     26    24    15    15
P4    3     5     17    14    9     9
Avg TAT: 14.25 | Avg WT: 7.75 | Avg RT: 7.75

Shortest Remaining Time First (SRTF)
PID   AT    BT    CT    TAT   WT    RT
P1    0     8     17    17    9     0
P2    1     4     5     4     0     0
P3    2     9     26    24    15    15
P4    3     5     10    7     2     2
Avg TAT: 13.00 | Avg WT: 6.50 | Avg RT: 4.25
```

5. Gantt Chart

```
FCFS:
+--------+----+---------+-----+
|   P1   | P2 |   P3    | P4  |
+--------+----+---------+-----+
0        8    12        21    26

SJF:
+--------+----+-----+---------+
|   P1   | P2 | P4  |   P3    |
+--------+----+-----+---------+
0        8    12    17        26

SRTF:
+--+------+------+-------+---------+
| P1|  P2  |  P4  |  P1   |   P3    |
+--+------+------+-------+---------+
0   1      5      10      17        26
```

6. Comparison

Algorithm	Avg TAT (ms)	Avg WT (ms)	Avg RT (ms)
FCFS	15.25	8.75	8.75
SJF (Non-Preemptive)	14.25	7.75	7.75
SRTF (Preemptive SJF)	**13.00**	**6.50**	**4.25**